🔷 1. End-to-End System Architecture (Logical View)

🧩 Core Layers

```
[ User / App / Agent Caller ]
        ↓
[ UISP Portal (UI + Chatbot) ]
        ↓
[ API Gateway + WAF + AuthBlue ]
        ↓
[ UISA Orchestrator (Agent Runtime) ]
        ↓
-----------------------------------------------
|   Agent Ecosystem Layer                      |
|  - Enterprise Agents                        |
|  - Security Domain Agents                   |
|  - MCP Tools (Local + Remote)               |
-----------------------------------------------
        ↓
-----------------------------------------------
|   Intelligence Layer                        |
|  - LLM (hosted / private)                   |
|  - RAG (Knowledge Base + API Specs)         |
|  - Policy Engine                           |
-----------------------------------------------
        ↓
-----------------------------------------------
|   Control Plane                              |
|  - Agent Registry                          |
|  - MCP Registry                            |
|  - Identity + AuthZ (Okta OAuth2.1)        |
-----------------------------------------------
        ↓
-----------------------------------------------
|   Data & Observability Layer                 |
|  - Conversation Store                      |
|  - Audit Logs                              |
|  - App Logs / Traces                       |
|  - JIRA Integration                        |
-----------------------------------------------
```

🔷 2. Actors

- 👤 End User (Employee / App Owner)

- 🤖 UISA (Primary Orchestrator Agent)
- 🤖 Domain Agents (Security domains)
- 🧰 MCP Tools (APIs, scripts, automation)
- 👨‍💼 Human SME (fallback / approval)
- 🔐 Identity Providers (AuthBlue + Okta)

🔷 3. End-to-End Flow (Success Path)

Step-by-step

1. User logs into UISP via **AuthBlue SSO**
2. Chatbot captures request
3. UISP → API Gateway (JWT propagated)
4. Request forwarded to UISA
5. UISA:
    - Retrieves:
        - Knowledge base (RAG)
        - Agent registry
        - MCP registry
    - Determines:
        - Domains involved
        - Required services
6. Intent validation (multi-step)
7. Dynamic orchestration:
    - Calls Domain Agent(s) via A2A (OAuth2.1)
    - Calls MCP tools via HTTP/STDIO
8. Validates inputs against API spec
9. Executes workflows
10. Creates JIRA tickets
11. Aggregates responses
12. Returns structured response to UI
13. Logs + audit + trace recorded

🔷 4. Failure & Exception Flows

Scenario	Handling
❌ No domain agent	Return fallback + knowledge suggestion
❌ Unauthorized	Deny with RBAC reason
❌ MCP failure	Retry + circuit breaker
❌ Intent mismatch	Ask clarification

❌ Partial domain success	Saga compensation or manual fallback
❌ LLM hallucination detected	Block + re-ground

🔷 5. Identity, AuthN & AuthZ (ALL Layers)

🔐 Layers

1. **User → UISP**
   - AuthBlue SSO (OIDC)
2. **UISP → UISA**
   - JWT propagation (user context)
   - mTLS (service trust)
3. **Agent-to-Agent (A2A)**
   - Okta OAuth 2.1
   - Token includes:
     - user context
     - agent identity
     - scopes
4. **Agent → MCP**
   - OAuth2.1 + SPIFFE/SPIRE (recommended)
   - STDIO: sandboxed + signed execution

🔐 Key Principle

👉 Always maintain **user-on-behalf-of (OBO)** identity propagation

🔷 6. Data Stores (Recommendations)

📘 Knowledge Base (RAG)

- **Vector DB**: Pinecone / Weaviate / OpenSearch Vector
- **Raw Docs**: S3 / Azure Blob
- Chunk + embed domain docs

📗 API Specs

- Store in:
  - Git (source of truth)
  - Sync to:
    - API Registry (SwaggerHub / Backstage)

📙 Agent Registry

- Backstage / Custom service registry
- Metadata:
  - capabilities
  - endpoints
  - auth requirements

📕 MCP Registry

- Service catalog (DB + API)
- Includes:
  - tool schema
  - protocol (HTTP/STDIO)
  - security classification

💬 Conversation Store

Best practice (hybrid):

Type	DB
Active conversations	Redis
Persistent	DynamoDB / MongoDB
Semantic retrieval	Vector DB

👉 Similar to ChatGPT:

- short-term memory → cache
- long-term → vector + document store

🔷 7. JIRA Strategy

❗ **Do NOT assign JIRA to agents**

✅ Best Practice:

- JIRA = human workflow + audit
- Agents:
  - create/update tickets
  - comment updates
- Use:
  - 1 parent ticket
  - child tickets per domain

🔷 8. Learning & Improvement

Continuous Learning Loop

1. Capture:
   - conversations
   - feedback
2. Pipeline:
   - human review
   - labeling
3. Feed into:
   - fine-tuning
   - prompt optimization
   - RAG updates

🔷 9. Human-in-the-Loop (HITL)

Required for:

- High-risk actions (access removal)
- Policy violations
- Low confidence LLM outputs
- Multi-domain conflicts

🔷 10. Audit & Compliance

Capture:

- user identity
- agent identity
- actions taken
- MCP tools invoked
- timestamps
- decisions

Store:

- Immutable logs (WORM storage)
- SIEM integration

🔷 11. Safety Layers (Where + What)

🔒 Layers

1. Input validation
2. Prompt injection detection
3. Output filtering
4. Policy enforcement engine
5. Tool access guardrails
6. LLM response validation

🔷 12. OWASP Protection

Cover:

- Prompt injection
- Data exfiltration
- Tool misuse
- Over-permissioned agents
- Insecure output handling

👉 Controls:

- strict tool allowlist
- scoped tokens
- sandbox execution
- output sanitization

🔷 13. Testing Strategy

Types

- Unit (agents/tools)
- Integration (A2A + MCP)
- Adversarial testing (prompt injection)
- Chaos testing (fail MCP/agent)
- Red teaming (LLM abuse)

🔷 14. Multi-Domain Transaction Handling

❗ Distributed transaction ≠ possible traditionally

Use:

- **Saga Pattern**
  - step orchestration
  - compensation actions

🔷 15. Trust Verification

Ensure:

- mTLS between services
- signed JWTs
- agent identity registry
- request provenance tracking

🔷 16. Notifications

- Event-driven (Kafka / SNS / EventBridge)
- Channels:
  - Email
  - Slack / Teams
  - Portal alerts

🔷 17. Observability (CRITICAL)

Must have:

- Distributed tracing (OpenTelemetry)
- Correlation ID (MANDATORY)

👉 Pass:

```
X-Transaction-ID
X-User-ID
X-Agent-ID
```

🔷 18. Metrics for Success

- Intent accuracy
- Completion rate
- SLA adherence
- User feedback score
- Hallucination rate

🔷 19. Grounding Strategy

- RAG (primary)
- API spec grounding
- tool-only execution
- multi-agent validation

🔷 20. Homegrown vs No-Code Agents

Aspect	Homegrown	No-code
Control	High	Low
Security	Strong	Limited
Speed	Medium	Fast
Customization	Full	Restricted

👉 Recommendation:

- Core → homegrown
- experimentation → no-code

🔷 21. Logging Strategy

Use:

- OpenTelemetry + ELK / Datadog

Log Types:

- app logs
- audit logs
- LLM logs (prompt/response masked)

🔷 22. Additional Threats (Beyond Hallucination)

- Tool abuse
- Agent impersonation
- Data poisoning
- Context leakage

🔷 23. Sandboxing (YES, REQUIRED)

Why:

- isolate MCP tools
- prevent code execution risks
- enforce resource limits

Options:

- containers (Docker)
- Firecracker microVMs

🔷 24. Recommended Tech Stack

Backend

- Java (Spring Boot) / Node.js

Agent Framework

- LangGraph / Semantic Kernel / custom

LLM

- Azure OpenAI / private LLM

Vector DB

- Pinecone / OpenSearch

Observability

- OpenTelemetry + Grafana

Identity

- Okta + AuthBlue

🔷 25. Final Key Design Principles

- 🔐 Zero Trust everywhere
- 🧠 LLM never trusted blindly
- 🔄 Everything observable
- 🔁 Retry + fallback everywhere
- 🧩 Modular agent architecture
- 🛑 Human gate for critical actions

✅ If you want next

I can:

- Create **draw.io XML diagram (ready to import)**
- Provide **sequence diagrams (A2A + MCP flows)**
- Generate **reference implementation (Spring Boot + Agent orchestration)**

Just tell me 👍

Good — this is one of the **most critical (and often misunderstood)** parts of secure agentic architecture.
Let's break down **User-On-Behalf-Of (OBO) identity propagation** in your UISP/

UISA system in a **practical, implementation-ready way**.

🔷 1. What "User-On-Behalf-Of (OBO)" REALLY Means

👉 Every downstream action must **carry the original user's identity and permissions**, even if executed by agents.
Instead of:

```
User → UISP → Agent → MCP → Action
           (loses user identity ❌)
```

You enforce:

```
User → UISP → UISA → Domain Agent → MCP Tool
        (user identity preserved end-to-end ✅)
```

🔷 2. Why This is NON-NEGOTIABLE

Without OBO:

Risk	Example
❌ Privilege escalation	Agent uses its own elevated access
❌ No accountability	Audit shows "agent did it"
❌ Compliance failure	No user traceability
❌ Unauthorized actions	Agent bypasses RBAC

👉 With OBO:

- Every action = **user-authorized**
- Agents = **delegated executors only**

🔷 3. OBO Identity Model (Your System)

Identity Chain

```
User (AuthBlue SSO)
   ↓
UISP (JWT: user identity)
   ↓
UISA (Token Exchange → OBO Token)
   ↓
Domain Agent (Scoped Token)
   ↓
MCP Tool (Final Execution with user context)
```

🔷 4. How to IMPLEMENT OBO (Step-by-Step)

🔐 Step 1: User Authentication

- User logs in via **AuthBlue (OIDC)**
- UISP gets:

```
{
  "sub": "user123",
  "email": "...",
  "roles": ["app_owner"],
  "groups": ["finance"],
  "scp": ["request:create"]
}
```

🔐 Step 2: Token Propagation (UISP → UISA)

- Forward JWT as:

```
Authorization: Bearer <user_token>
```

- Add:

```
X-User-Context: <base64 encoded claims>
```

🔐 Step 3: Token Exchange (CRITICAL)

UISA must NOT reuse the same token blindly.

👉 Use **OAuth 2.1 Token Exchange (RFC 8693)** via Okta

UISA calls Okta:

```
POST /token
grant_type=urn:ietf:params:oauth:grant-type:token-exchange
subject_token=<user_token>
requested_token_type=access_token
audience=domain-agent
```

Result:

👉 New **OBO token**:

```
{
  "sub": "user123",
  "act": {
    "sub": "uisa-agent"
  },
```

```
  "scope": "domain.read domain.write",
  "aud": "domain-agent"
}
```

🔐 Step 4: Agent-to-Agent (A2A)

When UISA calls Domain Agent:

```
Authorization: Bearer <OBO token>
```

Domain Agent sees:

- User = user123
- Acting agent = uisa-agent

👉 This is called **actor claim (act) pattern**

🔐 Step 5: Agent → MCP Tool

Repeat token exchange again:

- Domain Agent exchanges token for MCP audience

Result:

```
User = user123
Actor = domain-agent
Delegated by = uisa-agent
```

👉 Full chain preserved

🔷 5. Token Structure (Best Practice)

```
{
  "sub": "user123",           // actual user
  "act": {
    "sub": "uisa-agent",      // acting agent
    "act": {
      "sub": "domain-agent"   // nested delegation
    }
  },
  "scp": ["access:modify"],
  "aud": "mcp-tool",
  "iss": "okta"
}
```

🔷 6. Authorization Enforcement (VERY IMPORTANT)

At **every layer**, enforce:

✅ Checks

1. User RBAC
2. Agent permission
3. Scope validation
4. Resource-level access

Example (MCP Tool)

Before executing:

```
IF user NOT allowed → DENY
IF agent NOT allowed → DENY
IF scope missing → DENY
```

🔷 7. Trust Verification (How systems trust each other)

Use:

- mTLS between services
- JWT signature validation
- Token audience validation
- Agent registry verification

🔷 8. Audit Trail (What to Capture)

Every action must log:

```
{
  "user": "user123",
  "actor_chain": ["uisa-agent", "domain-agent"],
  "action": "grant_access",
  "resource": "db-prod",
  "timestamp": "...",
  "decision": "approved"
}
```

🔷 9. Common Mistakes (Avoid These)

Mistake	Impact
❌ Using service account token	breaks OBO
❌ Dropping user context	no audit

❌ Over-scoped tokens	security risk
❌ No audience validation	token replay

🔷 10. Advanced Enhancements

🔷 SPIFFE/SPIRE (Recommended)

- Strong workload identity
- Complements OAuth

🔷 Policy Engine

- Use OPA / Cedar

Example:

```
allow if user.role == "admin" AND action == "delete"
```

🔷 Token Downscoping

Each hop should:

- reduce permissions
- limit scope

🔷 11. Visual Summary

```
User (AuthBlue)
   ↓ JWT
UISP
   ↓ JWT
UISA
   ↓ OBO Token (Okta Token Exchange)
Domain Agent
   ↓ OBO Token (Downscoped)
MCP Tool
```

🔷 12. Final Recommendation (Enterprise Grade)

✅ MUST HAVE:

- OAuth 2.1 Token Exchange
- Actor (act) claim chaining
- Audience-restricted tokens
- Short-lived tokens (5–15 min)
- Full audit logging

✅ Bottom Line

👉 Agents should **never act on their own authority**

They must always act:
“On behalf of a user, with explicitly delegated, scoped permissions, fully traceable across the system.”

If you want next, I can:

- Create **sequence diagram for OBO flow (UISP → UISA → Domain → MCP)**
- Provide **Spring Boot + Okta token exchange sample code**
- Design **OPA policies for your use case**

Just tell me 👍